

BHARAT CYBER NYAY PORTAL

Cyber Crime Reporting and Case Tracking System

Project Title: Bharat Cyber Nyay Portal – Cyber Crime Reporting System **Subtitle:** A Full-Stack Web Application for Secure Cyber Crime Reporting, Evidence Management, and Case Tracking **Developer:** [Developer Name] **Course / Program:** [Course Name] **University / Institution:** [University Name] **Academic Year:** 2025–2026 **Submission Date:** March 2026

TABLE OF CONTENTS

Section	Title	Page
1	Abstract	3
2	Introduction	4
3	Problem Statement	6
4	Proposed Solution	7
5	System Architecture	8
6	Technology Stack	11
7	Features of the System	14
8	Cyber Crime Modules	17
9	Security Implementation	18
10	Database Design	21
11	User Interface Design	23
12	Workflow Explanation	25
13	Testing	27
14	Deployment	29
15	Limitations	31
16	Future Enhancements	32
17	Conclusion	33
18	References	34

SECTION 1 — ABSTRACT

The **Bharat Cyber Nyay Portal** is a full-stack, web-based cyber crime reporting and case management system designed to bridge the gap between aggrieved citizens and law enforcement authorities in the digital domain. The portal enables citizens of India to register cyber crime complaints online, upload supporting evidence, and monitor the progress of their cases through a transparent, role-based workflow.

The system is architected using modern web technologies — a **Next.js 14** frontend deployed on **Vercel**, a **Node.js/Express.js** REST API backend deployed on **Render**, and a **PostgreSQL** database provisioned through **Supabase**. Authentication is enforced through **JSON Web Tokens (JWT)** and passwords are secured through **bcrypt** hashing. Role-based access control segregates citizen, investigator, and administrator functions.

The primary purpose of this system is to provide a digitised, accessible, and accountable channel through which cyber crime victims can file formal complaints without the need to physically visit a police station. India recorded over 15.9 lakh cybercrime complaints on the National Cyber Crime Reporting Portal (NCRP) in 2023 alone, reflecting the immense volume and urgency for scalable digital solutions. This project directly addresses that need by offering a structured, evidence-preserving, multi-role case management workflow.

The portal also supports **anonymous reporting** for specific crime categories — such as Phishing, Fake Websites, and Social Media Harassment — acknowledging the sensitive nature of such complaints and the psychological barriers that often prevent victims from coming forward.

SECTION 2 — INTRODUCTION

2.1 Background of Cyber Crime in India

The proliferation of internet services, digital payment systems, and social media platforms across India has been accompanied by an equally significant rise in cyber criminal activity. From Unified Payments Interface (UPI) fraud and phishing scams to identity theft and cryptocurrency fraud, the digital landscape presents increasingly sophisticated threats to citizens, businesses, and government institutions alike.

According to the **Indian Cyber Crime Coordination Centre (I4C)** and the **National Crime Records Bureau (NCRB)**, India saw a 113% increase in cyber crime complaints between 2021 and 2023. Financial fraud alone accounted for over ₹10,319 crore in

reported losses in 2023. The most targeted demographics are senior citizens, students, and individuals with limited digital literacy.

The **National Cyber Crime Reporting Portal (NCRP)** operated by the Ministry of Home Affairs provides a mechanism for reporting cyber crimes. However, citizen-facing transparency, real-time case tracking, investigator-citizen communication, and evidence management remain under-developed in publicly available tooling.

2.2 Need for Digital Reporting Systems

Traditional complaint mechanisms require physical visits to police stations, involve paper-based processes, and offer minimal visibility to the complainant after the initial filing. The following challenges exist in the current landscape:

- Lack of a transparent, citizen-accessible case status tracking mechanism
- Absence of structured digital evidence submission and verification workflows
- Insufficient integration between evidence upload, case assignment, and investigation updates
- High abandonment rates due to social stigma and fear of identity exposure
- No analytics layer for authorities to identify emerging crime patterns geographically

A purpose-built digital system that combines case management, evidence preservation, role-based access, and anonymity options directly addresses all of the above shortcomings.

2.3 Objectives of the Project

The following objectives guided the design and development of the Bharat Cyber Nyay Portal:

1. **Provide a citizen-facing interface** for submitting detailed cyber crime reports with mandatory and optional metadata fields.
2. **Enable secure evidence upload** with file integrity verification using SHA-256 hashing.
3. **Implement role-based access control** distinguishing between three roles: Citizen, Investigator, and Administrator.
4. **Offer real-time case tracking** using a unique case/report identifier.
5. **Support anonymous reporting** for sensitive crime categories, reducing psychological barriers.
6. **Equip administrators and investigators** with dashboards, case assignment tools, and investigation note functionality.
7. **Provide an analytics dashboard** visualising crime distribution, monthly trends, geographic distribution, and financial loss data.

8. **Deploy the system cost-effectively** on modern cloud platforms using free-tier services suitable for a functional prototype.

SECTION 3 — PROBLEM STATEMENT

3.1 Deficiencies in Existing Reporting Mechanisms

Despite the existence of government portals for cyber crime reporting in India, several systemic limitations continue to reduce their effectiveness:

3.1.1 Opacity of Case Status Once a citizen files a report — whether online or at a police station — there is typically no transparent mechanism to track the progress of their case. Complainants frequently have no means to determine whether their report has been assigned, reviewed, or acted upon.

3.1.2 Inadequate Evidence Handling Physical evidence such as screenshots, transaction records, or communication logs is often submitted informally. No standardised digital process exists for verifying file integrity, securely storing evidence, or linking evidence records permanently to their corresponding case.

3.1.3 Absence of Investigator Collaboration Tools Investigators assigned to cyber crime cases lack a structured digital workspace where they can record investigation notes, track status changes, review uploaded evidence, and communicate updates. This gap leads to information silos and delays in case resolution.

3.1.4 No Structured Anonymous Pathway Many cyber crime victims — particularly those targeted in social media harassment, sextortion, or phishing attacks — are reluctant to file formal complaints due to fear of public exposure or social stigma. The absence of a formal anonymous reporting channel leads to significant under-reporting.

3.1.5 Limited Analytical Capability Law enforcement agencies and policy makers lack real-time, accessible analytics to identify which crime types are increasing, which geographic regions are most affected, and how systemic improvements can be targeted.

3.2 Summary of Identified Gaps

Gap	Impact
No case tracking for citizens	Citizens feel ignored; cases go unmonitored
Informal evidence submission	Evidence integrity is unverifiable
No investigator notes system	Investigations lack documentation trails

No anonymity option

Under-reporting of sensitive crimes

No analytics dashboard

Limited policy and resource planning capability

SECTION 4 — PROPOSED SOLUTION

4.1 Overview of the Portal

The **Bharat Cyber Nyay Portal** is proposed as a structured, multi-role web application that addresses all identified deficiencies. The portal is organised around a core workflow: a citizen files a crime report; the system assigns a unique report identifier; an administrator reviews and assigns the report to an investigator; the investigator records notes and updates case status; the citizen tracks the case using their identifier.

The system is conceived as a prototype demonstrating how a modern, accessible, and cost-effective citizen-facing cyber crime reporting system can be built and deployed.

4.2 Key Functionalities

Citizen-Facing Capabilities

- Secure registration and email/password login with JWT session management
- Report submission form capturing: victim details, crime type, date/time of incident, suspect information, financial loss, and geographic location
- Digital evidence upload (images, PDFs, text files) with automatic SHA-256 integrity hashing
- Case tracking using report ID, providing real-time status visibility
- Anonymous reporting for eligible crime categories

Investigator-Facing Capabilities

- Dedicated dashboard showing all assigned cases
- Access to full report details and associated evidence
- Ability to append investigation notes to cases
- Case status update functionality

Administrator-Facing Capabilities

- Full view of all reports with filtering (by crime type, status, investigator)
- Investigator management (register, view, manage investigator accounts)
- Case assignment — assigning or re-assigning reports to investigators
- Access to the analytics dashboard

System-Wide Capabilities

- Analytics dashboard with charts: crime type distribution, monthly filing trend, case status breakdown, geographic distribution, and aggregate financial loss
- Awareness resources page providing guidance on cyber safety
- Responsive UI across desktop and mobile viewports

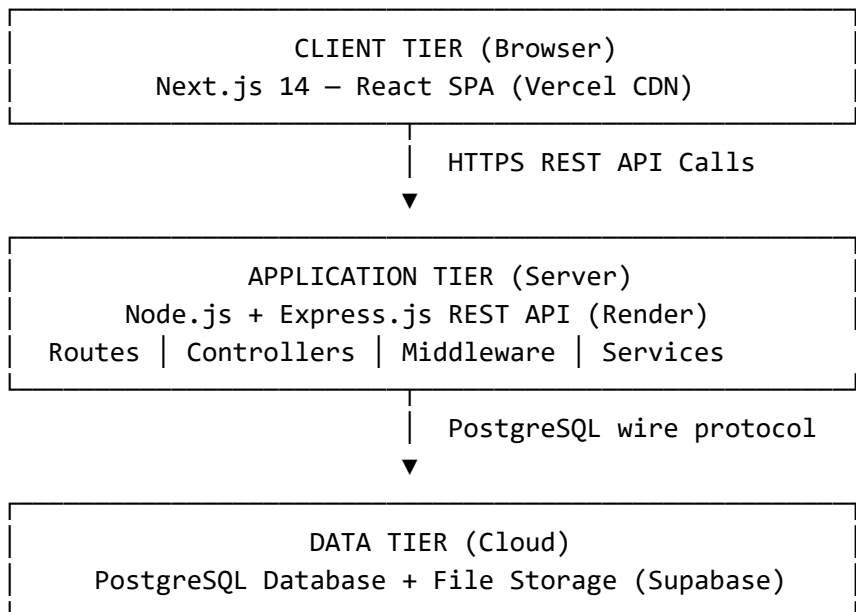
4.3 Benefits of the System

Benefit	Description
Accessibility	Citizens can report crimes from any device without visiting a police station
Transparency	Case tracking gives complainants visibility into their report's lifecycle
Evidence Integrity	SHA-256 hashing ensures uploaded files are tamper-evident
Accountability	Role-based access ensures data is only accessible to authorised parties
Anonymity	Sensitive crimes can be reported without identity disclosure
Scalability	Cloud-native deployment supports horizontal scaling as user load grows
Cost Efficiency	Entire stack is deployable on free-tier cloud services for prototyping

SECTION 5 — SYSTEM ARCHITECTURE

5.1 Architectural Overview

The Bharat Cyber Nyay Portal follows a **three-tier client-server architecture**:



5.2 Frontend Architecture

The frontend is a **Next.js 14** application using the App Router. It is a React-based Single Page Application (SPA) with server-side and client-side rendering capabilities.

Directory Structure

```

frontend/src/
├── app/                                # App Router pages
│   ├── layout.js                      # Root layout (Navbar, global styles)
│   ├── page.js                       # Public-facing homepage
│   ├── login/                        # Authentication page
│   ├── register/                    # Citizen registration page
│   ├── report/                      # Crime report submission page
│   ├── track/                       # Case tracking page (public)
│   ├── profile/                     # Citizen profile and own reports
│   ├── dashboard/                   # Role-aware main dashboard
│   │   ├── admin/                   # Admin-only panel (reports, investigator management)
│   │   └── analytics/               # Analytics dashboard
│   ├── resources/                   # Cyber awareness resources
│   └── unauthorized/                 # Access-denied boundary page
├── components/                       # Reusable React components
│   ├── Navbar.jsx
│   ├── ReportForm.jsx
│   ├── EvidenceUploader.jsx
│   ├── CaseTracker.jsx
│   ├── AdminReportsTable.jsx
│   ├── AdminCaseAssignment.jsx
│   ├── InvestigatorDashboard.jsx
│   ├── InvestigatorManagement.jsx
│   ├── AnalyticsCharts.jsx
│   ├── AuthForm.jsx
│   ├── ProfileSummary.jsx
│   └── ProtectedRoute.jsx
├── context/
│   └── AuthContext.js                # Global authentication state
└── lib/
    ├── api.js                       # Axios instance and API helper functions
    └── auth.js                       # Token storage and auth utilities

```

Key Frontend Design Decisions

- **AuthContext:** A React Context Provider wraps the entire application and provides authentication state (user object, token, login/logout methods) to all child components without prop drilling.
- **ProtectedRoute:** A higher-order component that guards routes based on the authenticated user's role. Unauthenticated or unauthorised access redirects to /login or /unauthorized.
- **api.js:** A centralised Axios instance pre-configured with the backend base URL and a request interceptor that automatically attaches the Bearer token from local storage to every outgoing request.
- **Recharts:** Used for the analytics dashboard charts, providing interactive bar charts, pie charts, and line graphs.

5.3 Backend Architecture

The backend is a **Node.js** REST API built with the **Express.js** framework. It follows the MVC (Model-View-Controller) architectural pattern separated into distinct concerns.

Directory Structure

```

backend/src/
├── server.js           # HTTP server entry point (port binding,
├── app.js             # Express app setup (middleware, routes)
├── config/
│   ├── env.js         # Centralised environment variable confi
│   ├── db.js          # Supabase admin client initialisation
│   └── supabase.js     # Supabase client configuration
├── routes/
│   ├── index.js       # Root router aggregating all sub-router
│   ├── authRoutes.js  # /api/auth
│   ├── reportRoutes.js # /api/reports
│   ├── caseRoutes.js  # /api/cases
│   ├── evidenceRoutes.js # /api/evidence
│   ├── analyticsRoutes.js # /api/analytics
│   └── adminRoutes.js  # /api/admin
├── controllers/
│   ├── authController.js
│   ├── reportController.js
│   ├── caseController.js
│   ├── evidenceController.js
│   ├── analyticsController.js
│   └── adminController.js
├── middleware/
│   ├── auth.js        # JWT verification middleware
│   ├── role.js        # Role-based access middleware
│   ├── optionalAuth.js # Middleware for optional authentication
│   ├── validate.js     # Zod schema validation middleware
│   └── errorHandler.js # Centralised error handler
├── models/
│   └── userModel.js    # User CRUD operations
├── validations/
│   ├── reportValidation.js # Zod schemas for report endpoints
│   ├── authValidation.js  # Zod schemas for auth endpoints
│   └── adminValidation.js  # Zod schemas for admin endpoints
├── utils/
│   ├── jwt.js          # JWT sign/verify helpers
│   ├── hash.js         # SHA-256 utility for file integrity
│   └── anonymousReporter.js # Anonymous user identity resolution
├── bootstrap/
│   └── seedPredefinedUsers.js # Seed admin and investigator account
└── db/
    └── schema.sql      # Full PostgreSQL schema definition

```

Express Middleware Pipeline

Every incoming HTTP request passes through the following middleware chain:

1. **helmet()** — Sets security-hardening HTTP headers (Content Security Policy, HSTS, X-Frame-Options, etc.)

2. **cors()** — Validates request origin against a configurable whitelist (FRONTEND_URLS env variable)
3. **express.json({ limit: '5mb' })** — Parses JSON request bodies with a size limit
4. **hpp()** — Prevents HTTP Parameter Pollution attacks
5. **rateLimit()** — Rate limiting: 300 requests per 15-minute window per IP
6. **Route handlers** — The request is dispatched to the appropriate router → controller
7. **validate(schema)** — Inline Zod validation middleware applied at the route level
8. **authenticate** — JWT verification (applied per route as required)
9. **role(...roles)** — Role authorisation guard
10. **errorHandler** — Catches all errors and returns a structured JSON error response

5.4 Database Architecture

The database layer uses **PostgreSQL** hosted on **Supabase**. The backend interacts with the database via two mechanisms:

1. **@supabase/supabase-js client** — Used for standard CRUD operations through Supabase's query builder API
2. **pg (node-postgres)** — Used for low-level raw SQL execution when Supabase's query builder is insufficient

The schema defines four primary tables (users, reports, evidence, case_notes) and five PostgreSQL stored functions (get_crime_distribution, get_monthly_trend, get_status_breakdown, get_financial_stats, get_reports_per_state) consumed by the analytics controller via Supabase's `.rpc()` method.

5.5 API Communication

The frontend communicates with the backend exclusively via **HTTPS REST API calls**. The Axios library manages all HTTP communication. A consistent JSON response structure is maintained across all endpoints:

```
{
  "status": 200,
  "data": { ... }
}
```

Error responses follow the format:

```
{  
  "message": "Descriptive error message",  
  "details": { ... }  
}
```

All write operations (POST, PATCH, PUT) validate their request body against a **Zod schema** before the controller logic executes, ensuring type safety and consistent error messaging.

SECTION 6 — TECHNOLOGY STACK

6.1 Frontend Technologies

6.1.1 Next.js 14 (React Framework)

Version: 14.2.15 **Purpose:** Frontend framework for building the user-facing application

Next.js is a React-based full-stack framework developed by Vercel. It was selected for this project for the following reasons:

- **App Router:** Next.js 14 introduces the App Router, providing a file-system-based routing model where each folder under `src/app/` corresponds to a URL route. This removes the need for a separate routing library.
- **Performance:** Automatic code splitting, lazy loading, and image optimisation reduce Time-to-Interactive metrics.
- **Server-Side Rendering (SSR) / Static Generation:** Pages can be rendered server-side for improved SEO and initial load performance, critical for a civic-facing portal.
- **Vercel Deployment Integration:** Being developed by the same team as Vercel, Next.js deploys with zero configuration on Vercel's global CDN.

6.1.2 React 18

Version: 18.3.1 **Purpose:** UI component model

React's declarative component model enables reusable, maintainable UI elements. React 18 introduces Concurrent Mode features including automatic batching, improving rendering performance under high interaction loads.

6.1.3 Tailwind CSS

Version: 3.4.14 **Purpose:** Utility-first CSS framework

Tailwind CSS enables rapid, consistent styling through utility classes directly in JSX. Benefits for this project include:

- No custom CSS file management
- Built-in responsive design utilities (md:, lg: prefixes)
- Consistent design token system (colours, spacing, typography)
- Minimal CSS bundle size in production

6.1.4 Axios

Version: 1.7.7 **Purpose:** HTTP client for API communication

Axios was preferred over the native fetch API because:

- Automatic JSON parsing and serialisation
- Request and response interceptors (used to inject JWT Bearer tokens)
- Better error handling with structured response objects

6.1.5 Recharts

Version: 2.12.7 **Purpose:** Data visualisation library for analytics charts

Recharts provides a composable, React-native charting library used to render the analytics dashboard's bar charts, pie charts, and line graphs for crime distribution, monthly trends, and financial loss statistics.

6.2 Backend Technologies

6.2.1 Node.js

Version: ≥18 (specified in engines field) **Purpose:** Server-side JavaScript runtime

Node.js was selected for its:

- Non-blocking, event-driven I/O model suitable for a request-intensive API
- Unified JavaScript language across frontend and backend
- Rich package ecosystem (npm)
- Compatibility with Render's free-tier hosting

6.2.2 Express.js

Version: 4.21.1 **Purpose:** Web application framework

Express.js is a minimal and flexible Node.js web framework. Its middleware-based pipeline model allowed fine-grained security and validation controls to be layered on

each route without tightly coupling concerns.

6.2.3 Zod

Version: 3.23.8 **Purpose:** Schema validation library

Zod provides TypeScript-first (and JavaScript-compatible) schema declaration and validation. Every API endpoint that consumes a request body validates it against a Zod schema before the controller executes. This eliminates an entire class of injection-adjacent bugs caused by unexpected or malformed input.

6.2.4 Multer

Version: 1.4.5-lts.1 **Purpose:** Multipart file upload middleware

Multer handles multipart/form-data requests for evidence file uploads. Configuration enforces:

- Maximum file size: **10 MB**
- Allowed MIME types: image/jpeg, image/png, image/webp, application/pdf, text/plain
- In-memory storage (buffer is immediately passed to Supabase Storage, preventing any server-side file persistence)

6.2.5 Helmet

Version: 8.0.0 **Purpose:** HTTP security header middleware

Helmet sets 11 security-hardening HTTP response headers including:

- Content-Security-Policy
- Strict-Transport-Security (HSTS)
- X-Frame-Options: DENY
- X-Content-Type-Options: nosniff

6.2.6 express-rate-limit

Version: 7.4.1 **Purpose:** API rate limiting

Limits each IP address to **300 requests per 15-minute window**, protecting the API from denial-of-service attacks and brute-force credential stuffing attempts.

6.2.7 hpp (HTTP Parameter Pollution)

Version: 0.2.3 **Purpose:** Parameter pollution prevention

Prevents attackers from injecting duplicate query string parameters that could bypass input validation or cause unexpected application behaviour.

6.3 Database and Storage

6.3.1 PostgreSQL (via Supabase)

Purpose: Relational database

PostgreSQL was selected for:

- **ACID compliance:** Every transaction affecting reports, evidence, and case notes is fully atomic and consistent.
- **UUID primary keys:** The users table uses UUID primary keys generated by PostgreSQL's pgcrypto extension (`gen_random_uuid()`), preventing sequential ID enumeration attacks.
- **CHECK constraints:** The role, status, and other enumerated fields are enforced at the database level, not only at the application layer.
- **Stored procedures:** Analytics queries are implemented as SECURITY DEFINER PostgreSQL functions, performing aggregations server-side without exposing raw table data to the API layer.
- **Foreign key constraints:** Relationships between tables are enforced with ON DELETE CASCADE and ON DELETE SET NULL rules.

6.3.2 Supabase

Purpose: Managed cloud platform providing PostgreSQL, Storage, and a JavaScript client SDK

Supabase was selected because:

- It provides a fully managed PostgreSQL instance on a **free tier** (suitable for this prototype)
 - Supabase Storage enables secure cloud storage for evidence files linked to their reports
 - The `@supabase/supabase-js` client library provides a clean query builder without requiring raw SQL for most operations
 - The service role key grants the backend admin-level database access required for cross-user operations (admin and investigator functions)
-

6.4 Authentication and Security Libraries

6.4.1 jsonwebtoken

Version: 9.0.2 **Purpose:** JWT signing and verification

JWTs (JSON Web Tokens) are used as the stateless session mechanism. Upon successful login, the server signs a JWT containing the user's `id` and `role` with a secret key. The client stores this token and includes it in the `Authorization: Bearer <token>` header on subsequent requests. The `auth.js` middleware verifies the token on every protected route.

6.4.2 bcryptjs

Version: 2.4.3 **Purpose:** Password hashing

All user passwords are hashed using the `bcrypt` algorithm with a cost factor of **12 rounds** before being stored in the database. Plaintext passwords are never stored. During login, `bcrypt.compare()` is used to verify the submitted password against the stored hash.

6.5 Deployment Platforms

6.5.1 Vercel (Frontend)

Vercel is the official hosting platform for Next.js applications. Key advantages:

- Zero-configuration continuous deployment from GitHub
- Global CDN with edge caching
- Automatic HTTPS certificate provisioning
- Free tier supports unlimited personal projects

6.5.2 Render (Backend)

Render is a modern cloud platform used to host the Node.js/Express API. Key advantages:

- Native support for Node.js applications
- Environment variable management
- Free-tier web services (with cold start on inactivity)
- Auto-deploy on push to the main branch

6.5.3 GitHub

The source code is version-controlled on GitHub. Both Vercel and Render are connected to the GitHub repository enabling automated CI/CD — every push to the main branch triggers a deployment.

6.6 Technology Stack Summary Table

Layer	Technology	Version	Purpose
Frontend Framework	Next.js	14.2.15	App Router, SSR, routing
UI Library	React	18.3.1	Component model
Styling	Tailwind CSS	3.4.14	Utility-first CSS
HTTP Client	Axios	1.7.7	API communication
Charts	Recharts	2.12.7	Analytics visualisation
Runtime	Node.js	≥18	Server-side JavaScript
API Framework	Express.js	4.21.1	REST API server
Validation	Zod	3.23.8	Schema validation
File Upload	Multer	1.4.5	Multipart handling
Database	PostgreSQL	15 (Supabase)	Relational data store
ORM/Client	@supabase/supabase-js	2.49.1	Database + storage client
Auth Tokens	jsonwebtoken	9.0.2	JWT session management
Password Hashing	bcryptjs	2.4.3	Secure credential storage
Security Headers	Helmet	8.0.0	HTTP hardening
Rate Limiting	express-rate-limit	7.4.1	DoS protection
Frontend Hosting	Vercel	—	CDN + CI/CD
Backend Hosting	Render	—	Cloud API hosting
Database Hosting	Supabase	—	Managed PostgreSQL + Storage
Version Control	GitHub	—	Source control + CI/CD

SECTION 7 — FEATURES OF THE SYSTEM

7.1 Citizen Features

7.1.1 User Registration and Login

Citizens can create a secure account by providing their name, email address, phone number, and password. The registration form validates all inputs through Zod schema checks on the backend. Passwords are hashed with bcrypt (cost factor 12) before storage. Upon successful login, the server returns a signed JWT valid for **one day**, which the frontend stores and uses for all subsequent authenticated requests.

7.1.2 Report a Cyber Crime

The report submission form is the primary function of the portal. It collects:

Field	Description
Victim Name	Full legal name of the victim
Email	Contact email of the complainant
Phone Number	Contact phone number
Crime Type	One of seven enumerated categories (see Section 8)
Description	Free-text description (minimum 20 characters)
Date and Time of Incident	ISO datetime of when the crime occurred
Suspect Details	Optional free-text field for suspect information
Financial Loss Amount	Numeric monetary amount (default: 0)
Location	Geographic location (state/city) where the crime occurred

The form validates all fields locally before submission. The backend applies the same Zod schema as a second layer of validation before the record is persisted to the database.

7.1.3 Evidence Upload

After a report is submitted, citizens can upload supporting evidence files through the Evidence Uploader component. The system enforces:

- **Permitted file types:** JPEG, PNG, WebP (images), PDF (documents), TXT (text logs)
- **Maximum file size:** 10 MB per file
- **Storage:** Files are uploaded to Supabase Storage under a path structured as `{reportId}/{timestamp}-{sanitisedFileName}`
- **Integrity:** The backend computes a **SHA-256 hash** of every uploaded file buffer before storing it. The hash is persisted in the evidence table, enabling future verification of file authenticity.

7.1.4 Case Tracking

Citizens receive a **numeric Report ID** upon successful complaint submission. They can enter this ID on the public `/track` page to retrieve the current status of their case without requiring login. The returned status is one of the five case states: **Submitted**, **Under Review**, **Investigation**, **Resolved**, or **Closed**.

7.1.5 Citizen Dashboard and Profile

Authenticated citizens have access to a profile dashboard that displays all reports they have previously filed, including the crime type, submission date, geographic location, and current status. This provides a complete personal complaint history.

7.1.6 Anonymous Reporting

The system supports anonymous report submission for the following crime categories:

- **Phishing**
- **Fake Websites**
- **Social Media Harassment**

When a report is submitted without an authenticated session for these categories, the system resolves a shared **anonymous reporter user ID** (a system-level user account) and associates the report with it. This allows basic report creation and evidence upload without requiring identity disclosure, while still maintaining data integrity by linking the report to a system-level user.

7.2 Administrator Features

7.2.1 All Reports View

Administrators have access to a filterable, searchable table of all reports in the system. Reports can be filtered by:

- Crime type
- Case status

- Assigned investigator

The table displays key report metadata including victim name, crime type, location, submission date, current status, and the assigned investigator.

7.2.2 Case Assignment

Administrators can assign a pending report to any available investigator through the `AdminCaseAssignment` component. The assignment updates the `assigned_investigator_id` foreign key in the `reports` table and automatically transitions the case status to **Under Review**.

7.2.3 Investigator Management

Administrators can view and manage the roster of investigator accounts. Investigator accounts are provisioned at system startup through the seed script (`seedPredefinedUsers.js`) or through the admin interface.

7.2.4 Analytics Dashboard Access

Administrators have exclusive access to the analytics dashboard, which provides visual insights across the entire dataset.

7.3 Investigator Features

7.3.1 Assigned Cases Dashboard

The Investigator Dashboard (`InvestigatorDashboard.jsx`) presents a filtered view showing only reports assigned to the logged-in investigator. This provides a focused workspace without exposing unrelated cases.

7.3.2 Case Notes

Investigators can append investigation notes to any assigned case through the case notes API. Notes are stored in the `case_notes` table with a timestamp and investigator identifier, creating a complete audit trail of investigative activity.

7.3.3 Case Status Updates

Investigators can update the status of assigned cases to reflect progressing investigation stages: **Under Review** → **Investigation** → **Resolved** or **Closed**.

7.3.4 Evidence Review

Investigators have access to uploaded evidence files and can view the associated SHA-256 hash to verify file integrity.

7.4 System-Wide Features

7.4.1 Analytics Dashboard

The analytics dashboard aggregates data through five PostgreSQL stored functions called via Supabase RPC:

Chart	Data Source Function	Description
Crime Distribution	<code>get_crime_distribution()</code>	Pie/bar chart of report count by crime type
Monthly Trend	<code>get_monthly_trend()</code>	Line chart of monthly report volume
Status Breakdown	<code>get_status_breakdown()</code>	Bar chart of reports by current status
Financial Loss	<code>get_financial_stats()</code>	Aggregate sum of reported financial losses
Reports Per State	<code>get_reports_per_state()</code>	Geographic distribution of reports

7.4.2 Cyber Awareness Resources

The `/resources` page provides educational content on common cyber crimes, best practices for digital safety, and guidance on what to do if victimised. This serves the dual purpose of reducing crime incidence and improving the quality of reports submitted.

7.4.3 Responsive Design

The entire portal is designed with Tailwind CSS's responsive utilities, ensuring full functionality and readability across desktop, tablet, and mobile screen sizes.

SECTION 8 — CYBER CRIME MODULES

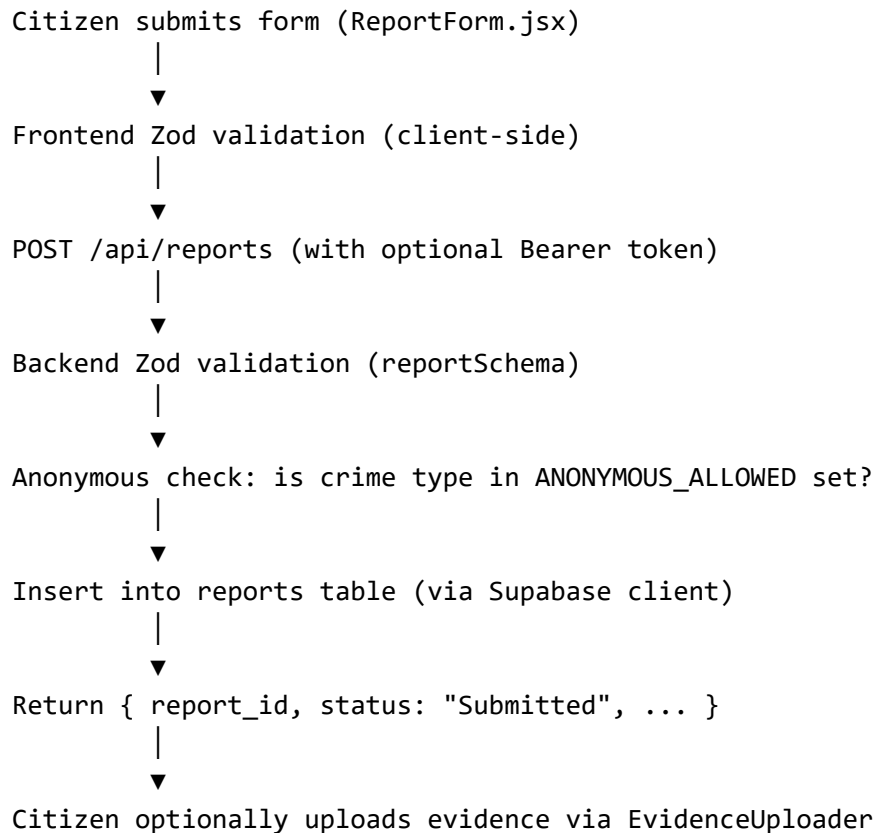
8.1 Supported Crime Categories

The system supports seven enumerated cyber crime categories, validated at both the frontend and backend using Zod schema enforcement:

#	Crime Category	Description
1	Phishing	Fraudulent emails, messages, or websites designed to steal login credentials or personal information
2	Online Fraud	E-commerce fraud, fake seller listings, advance fee fraud, and other internet-facilitated deceptions
3	UPI Scams	Fraudulent Unified Payment Interface (UPI) transactions, fake payment request scams, and digital wallet fraud
4	Social Media Harassment	Cyberbullying, impersonation, trolling, defamation, and unwanted contact via social media platforms
5	Identity Theft	Unauthorised use of a person's identity for financial gain, fraudulent applications, or criminal activities
6	Cryptocurrency Scams	Fake exchange platforms, rug-pull investments, Ponzi schemes, and token fraud in the cryptocurrency ecosystem
7	Fake Websites	Counterfeit e-commerce, banking, or government websites designed to deceive users into providing credentials or payments

8.2 Reporting Workflow

The end-to-end report submission process is as follows:



8.3 Evidence Handling System

The evidence handling pipeline was designed to maintain file integrity and chain-of-custody standards:

1. **Upload Trigger:** The citizen initiates upload via the EvidenceUploader component, selecting a file from their device.
2. **Client Validation:** File type and size are validated on the browser before sending.
3. **Multipart Transmission:** The file is transmitted as multipart/form-data to POST /api/evidence/:reportId.
4. **Multer Processing:** The backend Multer middleware intercepts the multipart request, storing the file in memory (no disk write).
5. **File Hash:** A SHA-256 hash of the file buffer is computed using Node.js's crypto module via sha256Buffer().
6. **Supabase Storage Upload:** The file buffer is uploaded to the Supabase Storage bucket (evidence-files) under a path of {reportId}/{timestamp}-{sanitisedName}.
7. **Public URL Retrieval:** A public download URL is generated from Supabase Storage for the uploaded file.
8. **Database Record:** An evidence record is inserted into the database containing the file URL, SHA-256 hash, MIME type, original filename, and upload timestamp.

9. **Response:** The backend returns the created evidence record including the public URL and hash.
-

SECTION 9 — SECURITY IMPLEMENTATION

Security was treated as a primary design concern throughout the development of the Bharat Cyber Nyay Portal, addressing multiple layers from HTTP transport to database access.

9.1 Authentication (JSON Web Tokens)

The portal uses **stateless JWT-based authentication**. The token payload contains the user's ID and role:

```
{
  "user_id": "uuid-string",
  "role": "citizen | investigator | admin",
  "iat": 1700000000,
  "exp": 1700086400
}
```

Token lifecycle:

- **Issued** on successful login by `authController.js`, signed with the `JWT_SECRET` environment variable
- **Transmitted** in the `Authorization: Bearer <token>` HTTP header
- **Verified** on every protected route by the `authenticate` middleware (`middleware/auth.js`)
- **Expiry:** Configurable via `JWT_EXPIRES_IN` environment variable (default: 1d)

The `authenticate` middleware additionally validates that the decoded token contains both `id` and `role` fields, rejecting tokens with an invalid payload structure with a 401 response.

9.2 Password Hashing (bcrypt)

bcryptjs is used to hash passwords before database storage:

```
const passwordHash = await bcrypt.hash(plainTextPassword, 12);
```


The cost factor of **12** means bcrypt performs 4,096 rounds of the Blowfish cipher before producing the hash. This makes brute-force attacks computationally expensive. During authentication, `bcrypt.compare()` is used — the plaintext password is **never stored** and **never logged**.

9.3 Input Validation (Zod Schemas)

Every endpoint that accepts request body data validates it with a Zod schema via the `validate(schema)` middleware. This provides several security properties:

- **Type coercion prevention:** Only expected fields of expected types are accepted
- **Length limits:** String fields have minimum and maximum length constraints
- **Enum enforcement:** Fields like `crimeType` and `status` are restricted to a defined set of values
- **Email format validation:** Email fields are validated against RFC 5322 format
- **Injection surface reduction:** Unexpected, malformed, or oversized inputs are rejected before reaching the controller or database layer

9.4 Role-Based Access Control (RBAC)

The system implements a three-tier RBAC model:

Role	Permissions
Citizen	Submit reports, upload evidence for own reports, view own reports, track any report by ID
Investigator	View and update all reports, upload evidence for any report, add case notes, update case status
Admin	All investigator permissions + user management, case assignment, analytics access

RBAC is enforced at the route level through the `role(...allowedRoles)` middleware. Protected routes specify which roles are permitted:

```
router.patch("/:id/assign", authenticate, role("admin"), assignInvestigator);
router.post("/:reportId/notes", authenticate, role("admin", "investigator"), addNotes);
```

The two predefined system accounts (Admin and Investigator) are seeded at application startup via `seedPredefinedUsers.js`, which checks for existence before creation to prevent duplicate seeding.

9.5 HTTP Security Headers (Helmet)

The `helmet()` middleware configures the following security headers on every response:

Header	Value / Effect
Content-Security-Policy	Restricts resource loading origins
Strict-Transport-Security	Forces HTTPS for a specified duration
X-Frame-Options	Prevents clickjacking via iframe embedding
X-Content-Type-Options	Prevents MIME-type sniffing
Referrer-Policy	Controls referrer information leakage
X-Permitted-Cross-Domain-Policies	Restricts Adobe Flash cross-domain requests

9.6 CORS (Cross-Origin Resource Sharing)

The backend enforces a strict CORS policy. The `origin` callback validates inbound request origins against the `FRONTEND_URLS` environment variable:

```
origin: (origin, callback) => {
  if (!origin || env.frontendUrls.includes(origin)) {
    return callback(null, true);
  }
  return callback(new Error("Not allowed by CORS"));
}
```

This prevents other websites from making credentialed API calls to the backend.

9.7 Rate Limiting

`express-rate-limit` enforces a limit of **300 requests per IP per 15 minutes** across all API endpoints. Requests exceeding this threshold receive a 429 Too Many Requests response. This mitigates:

- Brute-force login attacks
- Credential stuffing
- Denial-of-service via API flooding

9.8 File Upload Security

Evidence upload security is layered:

1. **MIME type whitelist:** Only 5 specific MIME types are accepted; all others are rejected by Multer's `fileFilter`
2. **File size limit:** 10 MB maximum enforced by Multer
3. **In-memory processing:** Files are processed in memory and streamed to Supabase Storage; no file writes occur on the API server
4. **Filename sanitisation:** Original filenames are sanitised by replacing non-alphanumeric characters with underscores before use in the storage path
5. **Ownership enforcement:** Citizens can only upload evidence for their own reports; investigators may upload for any report

9.9 Anonymous Reporting Security

Anonymous reports are associated with a shared system-level "anonymous reporter" user account rather than being stored with null user IDs. This approach:

- Maintains database referential integrity (the `user_id` FK is always satisfied)
 - Restricts anonymous reporting to a limited set of crime categories
 - Prevents anonymous users from accessing reports through the authenticated tracking interface
-

SECTION 10 — DATABASE DESIGN

10.1 Schema Overview

The database uses **PostgreSQL** with the `pgcrypto` extension for UUID generation. The schema consists of four tables and seven indices.

10.2 Table Definitions

10.2.1 users Table

Stores all user accounts regardless of role.

Column	Type	Constraints	Description
<code>id</code>	UUID	PRIMARY KEY, DEFAULT <code>gen_random_uuid()</code>	Unique user identifier
<code>name</code>	VARCHAR(120)	NOT NULL	Full name
<code>email</code>	VARCHAR(255)	UNIQUE, NOT NULL	Login email

phone	VARCHAR(20)	—	Contact phone number
password_hash	TEXT	NOT NULL	bcrypt hash of password
role	VARCHAR(20)	NOT NULL, CHECK IN ('citizen', 'investigator', 'admin')	User role
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Account creation timestamp

10.2.2 reports Table

The central entity of the system. Stores one record per crime report.

Column	Type	Constraints	Description
report_id	BIGSERIAL	PRIMARY KEY	Auto-incrementing numeric report ID
user_id	UUID	FK → users(id) ON DELETE CASCADE	Report submitter
victim_name	VARCHAR(120)	NOT NULL	Name of the victim
email	VARCHAR(255)	NOT NULL	Victim's email
phone_number	VARCHAR(20)	NOT NULL	Victim's phone
crime_type	VARCHAR(80)	NOT NULL	Type of cyber crime
description	TEXT	NOT NULL	Detailed description of the incident
incident_datetime	TIMESTAMP	NOT NULL	Date and time of the crime
suspect_details	TEXT	—	Optional suspect

			information
financial_loss_amount	NUMERIC(12,2)	NOT NULL, DEFAULT 0	Reported financial loss in rupees
location	VARCHAR(120)	NOT NULL	Geographic location
status	VARCHAR(30)	NOT NULL, DEFAULT 'Submitted', CHECK IN (...)	Case status
assigned_investigator_id	UUID	FK → users(id) ON DELETE SET NULL	Assigned investigator
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Submission timestamp
updated_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Last update timestamp

10.2.3 evidence Table

Stores metadata about every uploaded evidence file.

Column	Type	Constraints	Description
evidence_id	BIGSERIAL	PRIMARY KEY	Evidence record identifier
report_id	BIGINT	FK → reports(report_id) ON DELETE CASCADE	Parent report
file_url	TEXT	NOT NULL	Public URL of stored file
file_hash	VARCHAR(64)	NOT NULL	SHA-256 hash of file content
mime_type	VARCHAR(120)	NOT NULL	MIME type of the file
original_name	VARCHAR(255)	NOT NULL	Original filename

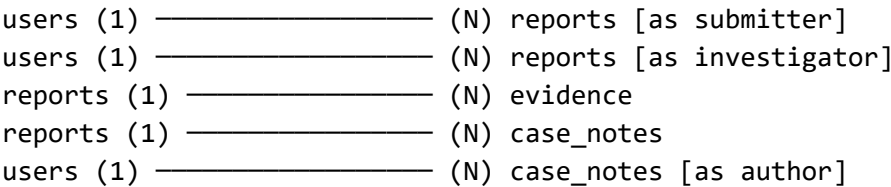
upload_time	TIMESTAMP	NOT NULL, DEFAULT NOW()	Upload timestamp
-------------	-----------	-------------------------	------------------

10.2.4 case_notes Table

Stores investigation notes appended by investigators.

Column	Type	Constraints	Description
note_id	BIGSERIAL	PRIMARY KEY	Note identifier
report_id	BIGINT	FK → reports(report_id) ON DELETE CASCADE	Associated report
investigator_id	UUID	FK → users(id) ON DELETE CASCADE	Note author
note_text	TEXT	NOT NULL	Note content
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()	Timestamp

10.3 Entity Relationship Summary



10.4 Database Indices

Seven composite indices were defined to optimise query performance:

Index	Column(s)	Purpose
idx_reports_status	status	Filter reports by case status
idx_reports_crime_type	crime_type	Filter by crime category
idx_reports_location	location	Geographic distribution

		queries
idx_reports_created_at	created_at	Time-ordered listing and trend analysis
idx_reports_assigned_investigator	assigned_investigator_id	Fetch reports by assigned investigator
idx_evidence_report_id	report_id	Evidence lookup per report
idx_case_notes_report_id	report_id	Case notes lookup per report

10.5 Stored Procedures

Five SECURITY DEFINER PostgreSQL functions perform analytics aggregations:

Function	Returns	Description
get_crime_distribution()	(label, value)	Count of reports per crime type
get_monthly_trend()	(month, reports)	Monthly report volume
get_status_breakdown()	(label, value)	Report count per status
get_financial_stats()	(total_loss)	Sum of all reported financial losses
get_reports_per_state()	(state, reports)	Report count per location

SECTION 11 — USER INTERFACE DESIGN

11.1 Design Principles

The portal's user interface was designed with three guiding principles:

1. **Clarity:** All calls to action are prominently positioned and labelled unambiguously. Citizens should be able to navigate from the homepage to filing a report in two clicks.
2. **Accessibility:** High-contrast colours, readable typography, and keyboard-navigable elements ensure the portal is usable by citizens with varying digital literacy levels.
3. **Trust:** A civic, professional visual tone — reinforced by a shield motif, formal typography, and a constrained colour palette — communicates that the portal is a credible government-aligned service.

11.2 Homepage (/)

The homepage presents an immediate, unambiguous call to action with three primary buttons:

- **File a Report** — Direct link to the report submission form
- **Report Anonymously** — Same form with awareness that only certain categories permit anonymity
- **Track Case Status** — Link to the public case tracker

A secondary section below the hero grid presents key stats or feature cards explaining the portal's purpose and security guarantees.

11.3 Report Submission Form (/report)

The report form uses a multi-field layout collecting all report metadata. Key UI considerations:

- Required fields are marked with asterisks
- Crime type is a dropdown with all seven supported categories
- Date/time picker for incident datetime
- Character count indicator on the description field (minimum 20 characters)
- Financial loss defaults to 0 (many crimes do not have direct monetary impact)

11.4 Evidence Uploader Component

The `EvidenceUploader.jsx` component renders after a report is submitted, showing the assigned report ID alongside a file drop-zone accepting the permitted MIME types. Upload progress and confirmation feedback are displayed inline.

11.5 Admin Dashboard (/dashboard/admin)

The admin panel presents:

- A tabular view of all reports with sortable columns
- Inline dropdowns for updating case status
- Investigator assignment selector per report row
- A separate "Investigators" tab for managing investigator accounts

11.6 Analytics Dashboard (/dashboard/analytics)

Using Recharts, the analytics page presents five visualisation panels:

- **Crime Type Distribution:** Horizontal bar chart
- **Monthly Trend:** Line chart with area fill
- **Status Breakdown:** Pie chart with legend
- **Total Financial Loss:** KPI card with formatted rupee value
- **Reports Per State:** Bar chart sorted by report volume

11.7 Navbar

The `Navbar.jsx` component is rendered in the root layout, appearing on every page. It displays:

- Portal logo and name
- Navigation links varying by authentication state and role
- Login/logout button
- Current user's name when authenticated

11.8 Responsive Layout

All pages use Tailwind CSS's `md:` and `lg:` responsive breakpoints. On mobile viewports:

- Navigation collapses to an appropriate compact layout
- Multi-column grids reorganise into single columns
- Form layouts stretch to full width

SECTION 12 — WORKFLOW EXPLANATION

12.1 End-to-End Case Lifecycle

The following describes the complete workflow for a cyber crime case from initiation to resolution.

Step 1 — Citizen Registration (First-Time Users)

1. Citizen visits the portal and clicks "Register"
2. Completes the registration form with name, email, phone, and password
3. The backend validates inputs, hashes the password, and creates the user record
4. The system issues a JWT and the citizen is redirected to their profile dashboard

Step 2 — Report Submission

1. Authenticated citizen navigates to /report
2. Fills the crime report form with all required details
3. Client-side validation checks all fields
4. Report is submitted via POST /api/reports with the Bearer token in the header
5. Backend validates the request body, confirms the user exists, inserts the report with status Submitted
6. The backend returns the created report object including the report_id
7. The citizen is shown their Report ID and prompted to upload evidence

Step 3 — Evidence Upload

1. Citizen selects a file in the EvidenceUploader component
2. File is submitted via POST /api/evidence/:reportId as multipart/form-data
3. Backend verifies file type, size, and ownership
4. File is streamed to Supabase Storage; SHA-256 hash is computed
5. An evidence record is created in the evidence table
6. The citizen receives confirmation with the file URL and integrity hash

Step 4 — Admin Review and Assignment

1. Admin logs into the portal and navigates to the admin dashboard
2. All reports with status Submitted are visible in the reports table
3. Admin reviews the report details
4. Admin selects an available investigator from the assignment dropdown
5. The backend processes PATCH /api/reports/:id/assign, updating assigned_investigator_id and setting status to Under Review

Step 5 — Investigation

1. Assigned investigator logs in and views their assigned cases in the Investigator Dashboard
2. Investigator reviews the report details, associated evidence, and file integrity hashes
3. Investigator progresses the case status to Investigation via PATCH /api/reports/:id/status

- Investigator appends investigation notes via POST `/api/cases/:reportId/notes` as work proceeds

Step 6 — Case Resolution

- Upon completing the investigation, the investigator or admin updates the status to `Resolved` or `Closed`
- The `updated_at` timestamp on the report is refreshed

Step 7 — Citizen Tracking

At any point from Step 2 onwards, the citizen can visit `/track`, enter their Report ID, and view the current case status. This provides a continuous feedback loop without requiring login.

12.2 Anonymous Reporting Workflow

For anonymous submissions:

- Citizen visits `/report` without logging in
- Selects a crime type from the anonymous-eligible set (Phishing, Fake Websites, Social Media Harassment)
- Completes and submits the form
- Backend resolves the anonymous reporter user ID via `getAnonymousReporterId()`
- Report is created and associated with the anonymous system user
- A Report ID is returned — the citizen can use this to track the case on `/track`

SECTION 13 — TESTING

13.1 Functional Testing

Functional testing was conducted to verify that each feature behaves as per specification.

13.1.1 Authentication Testing

Test Case	Input	Expected Output	Result
Register with valid data	Valid email, strong password	201 Created, JWT returned	Pass
Register with duplicate email	Existing email	409 Conflict	Pass

Login with correct credentials	Valid email and password	200 OK, JWT returned	Pass
Login with wrong password	Valid email, incorrect password	401 Unauthorized	Pass
Access protected route without token	No Authorization header	401 Unauthorized	Pass

13.1.2 Report Submission Testing

Test Case	Input	Expected Output	Result
Submit valid report (authenticated)	All required fields	201 Created, report object	Pass
Submit with invalid crime type	"RandomType"	400 Bad Request (Zod error)	Pass
Submit anonymous – eligible type	Phishing, no auth	201 Created	Pass
Submit anonymous – ineligible type	"UPI scams", no auth	403 Forbidden	Pass
Submit with missing description	Empty description field	400 Bad Request	Pass
Submit with description too short	10-char description	400 Bad Request	Pass

13.1.3 Evidence Upload Testing

Test Case	Input	Expected Output	Result
Upload valid JPEG	image/jpeg, < 10 MB	201 Created, evidence record	Pass
Upload unsupported type	.exe file	400 Bad Request	Pass
Upload oversized file	> 10 MB JPEG	400 Bad Request	Pass
Upload for another user's report	Citizen B uploading to Citizen A's report	403 Forbidden	Pass
Upload as investigator for any report	Investigator role	201 Created	Pass

13.1.4 Role-Based Access Testing

Test Case	Role	Route	Expected	Result
Citizen accessing admin panel	citizen	/dashboard/admin	Redirect to /unauthorized	Pass
Investigator accessing analytics	investigator	/dashboard/analytics	Access denied	Pass
Admin accessing all reports	admin	/api/admin/reports	200 OK	Pass
Investigator updating status	investigator	PATCH /api/reports/:id/status	200 OK	Pass
Citizen updating any report status	citizen	PATCH /api/reports/:id/status	403 Forbidden	Pass

13.2 Security Testing

13.2.1 Input Injection Testing

SQL injection attempts through report fields (e.g., `' ; DROP TABLE reports; --`) were tested. The Supabase client's parameterised query builder prevents SQL injection by design — inputs are never interpolated directly into SQL strings.

XSS (Cross-Site Scripting) payload strings (e.g., `<script>alert(1)</script>`) submitted through form fields are stored as plain strings and rendered by React's JSX engine, which escapes HTML by default, preventing execution.

13.2.2 Authentication Boundary Testing

- Modified JWT tokens (altered payload, invalid signature) are rejected with 401
- Expired tokens are rejected with 401
- Tokens from one environment (development) are rejected in another (production) due to different `JWT_SECRET` values

13.2.3 Rate Limiting Verification

Automated rapid requests to the `/api/auth/login` endpoint trigger the rate limiter after exceeding 300 requests within 15 minutes, returning 429 Too Many Requests.

13.3 API Testing

API endpoints were tested using **Postman** and manual HTTP requests. A comprehensive Postman collection was developed covering:

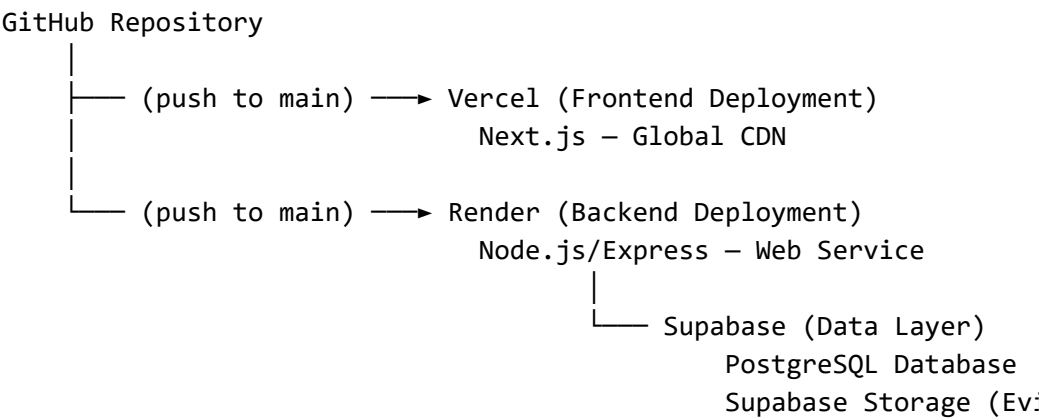
- All authentication endpoints
- Report CRUD operations
- Evidence upload with multipart payloads
- Case management endpoints (assignment, status update, notes)
- Analytics aggregation endpoints
- Admin management endpoints

All endpoints returned appropriate HTTP status codes, JSON response bodies, and error messages for both success and failure scenarios.

SECTION 14 — DEPLOYMENT

14.1 Architecture Overview

The system is deployed across three cloud platforms in a decoupled architecture:



14.2 Frontend Deployment (Vercel)

Setup Steps:

1. Connect the GitHub repository to Vercel
2. Specify frontend/ as the root directory
3. Set the build command to `next build`
4. Configure environment variables:

Variable	Description
----------	-------------

`NEXT_PUBLIC_API_URL` Backend API URL (Render deployment URL)

Vercel automatically provisions HTTPS, optimises the Next.js build, and deploys to its global edge network. Every `git push` to the main branch triggers a new deployment.

vercel.json Configuration:

The `vercel.json` file specifies any custom routing rewrites or headers needed for the Next.js application, including handling of client-side routing for the App Router.

14.3 Backend Deployment (Render)

Setup Steps:

1. Connect the GitHub repository to Render
2. Create a new Web Service pointing to `backend/`
3. Set the start command to `node src/server.js`
4. Configure environment variables:

Variable	Description
<code>SUPABASE_URL</code>	Supabase project URL
<code>SUPABASE_SERVICE_ROLE_KEY</code>	Service role key (admin Supabase access)
<code>JWT_SECRET</code>	Secret key for JWT signing
<code>JWT_EXPIRES_IN</code>	Token expiry (e.g., 1d)
<code>FRONTEND_URLS</code>	Comma-separated list of allowed frontend origins
<code>PREDEFINED_ADMIN_EMAIL</code>	Seed admin email
<code>PREDEFINED_ADMIN_PASSWORD</code>	Seed admin password
<code>PREDEFINED_ADMIN_NAME</code>	Seed admin name
<code>PREDEFINED_INVESTIGATOR_EMAIL</code>	Seed investigator email
<code>PREDEFINED_INVESTIGATOR_PASSWORD</code>	Seed investigator password
<code>PREDEFINED_INVESTIGATOR_NAME</code>	Seed investigator name
<code>SUPABASE_STORAGE_BUCKET</code>	Evidence files bucket name

render.yaml:

The `render.yaml` file in the `backend/` directory declaratively defines the Render web service configuration for Infrastructure-as-Code style deployment.

14.4 Database Setup (Supabase)

1. Create a new Supabase project
2. Navigate to the SQL Editor in the Supabase dashboard
3. Execute the full `backend/src/db/schema.sql` script, which creates all tables, indices, and stored functions
4. Copy the project URL and service role key to the Render environment variables

14.5 AI/CD (Continuous Integration and Deployment)

Both Vercel and Render are configured for **automatic deployment** triggered by pushes to the main branch of the GitHub repository. This provides a rudimentary CI/CD pipeline:

1. Developer pushes code changes to GitHub
2. Vercel detects the push, builds the frontend, and deploys to the CDN
3. Render detects the push, installs backend dependencies, and restarts the web service
4. The seed script (`seedPredefinedUsers.js`) runs at startup, creating predefined user accounts if they do not already exist

14.6 Local Development Setup

Prerequisites:

- Node.js ≥ 18
- npm ≥ 9
- A Supabase project with the schema applied

Backend:

```
cd backend
npm install
# Create .env file with required variables
npm run dev          # Starts nodemon watcher on src/server.js
```

Frontend:


```
cd frontend
npm install
# Create .env.local with NEXT_PUBLIC_API_URL
npm run dev          # Starts Next.js development server on port 3000
```

SECTION 15 — LIMITATIONS

15.1 Free-Tier Service Constraints

The current deployment relies on free tiers of cloud platforms, which impose the following limitations:

Platform	Limitation	Impact
Render (Free Tier)	Web service sleeps after 15 minutes of inactivity	API has a cold start delay of 30–60 seconds after idle periods
Supabase (Free Tier)	500 MB database storage; 1 GB file storage	Limits the scale of production use
Vercel (Free Tier)	Bandwidth and build minute limits	Suitable for prototype traffic volumes only

15.2 No Real Government Integration

This portal is an independent prototype. It does not integrate with:

- The National Cyber Crime Reporting Portal (NCRP) operated by the Ministry of Home Affairs
- Any state police FIR filing system
- Aadhaar-based identity verification

Reports submitted through this portal do not constitute formal legal complaints recognised by law enforcement.

15.3 Limited Real-Time Capabilities

The current implementation is request-response based. There is no WebSocket or Server-Sent Events (SSE) implementation, meaning:

- Citizens must manually refresh the case tracking page to see status updates
- Investigators do not receive push notifications for new case assignments
- Administrators do not receive real-time alerts for new reports

15.4 No Email Notification System

The system does not currently send email notifications:

- No confirmation email upon report submission
- No alert to the citizen when case status changes
- No notification to investigators upon assignment

15.5 Single Jurisdiction Scope

The location field accepts free text input for Indian states and cities. There is no structured administrative hierarchy or mapping. Geographic analytics are limited to whatever text citizens enter in the location field.

15.6 No Mobile Application

The portal is a responsive web application only. A dedicated iOS or Android mobile application with push notification and camera/gallery integration for evidence upload is not available in the current version.

SECTION 16 — FUTURE ENHANCEMENTS

16.1 AI-Based Fraud Detection

Integration of a machine learning model to:

- Automatically classify the crime type from the description text, reducing manual categorisation errors
- Flag suspicious or duplicate reports (potential fraud submissions)
- Predict investigation priority based on factors such as financial loss amount, crime type, and historical resolution time

16.2 Mobile Application

Development of a Progressive Web App (PWA) or native mobile application (React Native) that:

- Enables camera-based evidence capture directly from a smartphone
- Delivers push notifications for case status updates
- Supports offline report drafting with sync on connectivity restoration

16.3 Real-Time Status Notifications

Implementation of WebSocket ([Socket.io](#)) or Server-Sent Events for:

- Instant case status update notifications pushed to the citizen's browser
- Real-time investigator assignment alerts
- Admin dashboard live updates for new incoming reports

16.4 Email and SMS Notification System

Integration with services such as SendGrid (email) or Twilio/Msg91 (SMS) to:

- Send case submission confirmation with Report ID
- Notify citizens of every status transition
- Alert investigators of new assignments

16.5 Aadhaar / DigiLocker Identity Verification

Integration with India's digital identity infrastructure to:

- Verify complainant identity for high-severity complaints
- Reduce fraudulent or duplicate reports
- Enable legally valid digital signatures on submitted reports

16.6 Banking and Payment System Integration

For UPI scam and online fraud cases:

- API integration with payment platforms to retrieve transaction history as structured evidence
- Ability to flag specific UPI IDs or virtual payment addresses associated with reports

16.7 Dark Web Monitoring Alerts

A background service that:

- Monitors dark web marketplaces and forums for data matching leaked credentials or stolen identity documents
- Automatically creates a preliminary report when a citizen's data appears in a breach

16.8 Multi-Language Support

Localisation of the portal into all 22 Indian scheduled languages using an i18n library (such as `next-intl`), significantly expanding accessibility to non-English-speaking citizens.

16.9 FIR Integration

Coordination with the Crime and Criminal Tracking Network & Systems (CCTNS) to:

- Allow verified reports to be automatically pre-populated into the FIR filing workflow at the relevant police station
- Return an official complaint acknowledgement number from the government system

SECTION 17 — CONCLUSION

17.1 Summary

The Bharat Cyber Nyay Portal is a comprehensive, full-stack web application developed to address the growing need for accessible, transparent, and secure cyber crime reporting in India. The portal successfully implements a structured, role-based workflow connecting citizens, investigators, and administrators through a cohesive set of features covering report submission, evidence management, case tracking, investigation tooling, and analytical monitoring.

The system was built using a modern, industry-standard technology stack — Next.js and Tailwind CSS on the frontend, Node.js/Express.js on the backend, and PostgreSQL on Supabase for data persistence — all deployable on free-tier cloud infrastructure, making the prototype both cost-effective and scalable.

17.2 Technical Achievements

The following technical goals were successfully achieved:

- **Stateless JWT authentication** with role-based access control across three user roles
- **SHA-256 evidence file hashing** providing a chain-of-custody mechanism for uploaded files
- **Anonymous reporting** supporting underserved victim demographics
- **PostgreSQL stored functions** enabling server-side analytics aggregations via Supabase RPC
- **Multi-layer security architecture** combining Helmet headers, CORS policy, HPP protection, rate limiting, Zod validation, bcrypt hashing, and RBAC
- **Cloud-native deployment** with automated CI/CD on Vercel and Render

17.3 Social Impact and Usefulness

The Bharat Cyber Nyay Portal demonstrates that a modern, citizen-centric digital reporting system can be built with widely available open-source tooling and deployed at negligible cost. If adopted at scale or integrated with government systems, such a portal could:

- Reduce the technical and psychological barriers to reporting cyber crimes
- Increase accountability and transparency in case handling
- Enable data-driven policy decisions through aggregated analytics
- Serve as a foundation for an officially recognised national cyber crime case management platform

The project reflects a commitment to leveraging technology for civic good and represents a meaningful contribution to the discourse on digital governance in India.

SECTION 18 — REFERENCES

18.1 Government and Statistical Sources

1. Ministry of Home Affairs, Government of India. (2023). *Annual Report on Cyber Crime Statistics: National Cyber Crime Reporting Portal (NCRP)*. Retrieved from <https://www.mha.gov.in>
2. Indian Cyber Crime Coordination Centre (I4C). (2023). *Cyber Crime Annual Report*. Ministry of Home Affairs, India.
3. National Crime Records Bureau (NCRB). (2023). *Crime in India 2022 Report*. Ministry of Home Affairs, India.
4. CERT-In (Indian Computer Emergency Response Team). (2024). *Annual Report 2023*. Ministry of Electronics and Information Technology.

18.2 Technology Documentation

5. Vercel, Inc. (2024). *Next.js 14 Documentation*. <https://nextjs.org/docs>
6. OpenJS Foundation. (2024). *Node.js v18 Documentation*. <https://nodejs.org/docs/latest-v18.x/api/>
7. OpenJS Foundation. (2024). *Express.js 4.x API Reference*. <https://expressjs.com/en/4x/api.html>
8. Supabase, Inc. (2024). *Supabase JavaScript Client Documentation*. <https://supabase.com/docs/reference/javascript>
9. The PostgreSQL Global Development Group. (2024). *PostgreSQL 15 Documentation*. <https://www.postgresql.org/docs/15/>
10. Colin McDonnell. (2024). *Zod Schema Validation Documentation*. <https://zod.dev>

11. Helmetjs contributors. (2024). *Helmet.js Security Headers Middleware*.
<https://helmetjs.github.io>
12. Auth0, Inc. (2024). *jsonwebtoken npm Package Documentation*.
<https://github.com/auth0/node-jwebtoken>
13. dcodeIO. (2024). *bcryptjs npm Package Documentation*.
<https://github.com/dcodeIO/bcrypt.js>
14. Recharts Team. (2024). *Recharts Documentation*. <https://recharts.org/en-US>
15. Adam Wathan et al. (2024). *Tailwind CSS v3 Documentation*.
<https://tailwindcss.com/docs>

18.3 Security References

16. OWASP Foundation. (2023). *OWASP Top 10: Web Application Security Risks*.
<https://owasp.org/www-project-top-ten/>
17. RFC 7519 — *JSON Web Token (JWT)*. Internet Engineering Task Force (IETF), 2015.
18. NIST. (2017). *Digital Identity Guidelines: Special Publication 800-63B*. National Institute of Standards and Technology.
19. Provos, N., & Mazières, D. (1999). *A Future-Adaptable Password Scheme*. USENIX Annual Technical Conference.

18.4 Academic/General References

20. Sharma, A., & Gupta, R. (2022). *Cyber Crime in India: Trends, Challenges and Countermeasures*. International Journal of Computer Applications, 184(14), 1–8.
21. Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures* (Doctoral dissertation, University of California, Irvine). [REST Architecture]

End of Document

Document Version: 1.0 **Total Sections:** 18 **Status:** Complete — Ready for PDF Export
Project: Bharat Cyber Nyay Portal — Cyber Crime Reporting System
